

Bloque 4 — Mini-especificaciones funcionales y técnicas

Facturación y cobranza: F001 a F007

1. Propósito del bloque

Este bloque construye las funciones principales para:

- Reutilizar los clientes existentes.
- Administrar contactos de facturación y cobranza.
- Registrar o importar facturas de venta.
- Consultar facturas.
- Calcular su estado de cobranza.
- Registrar pagos recibidos o informados.
- Distribuir un pago entre varias facturas.
- Permitir que una factura sea cubierta por varios pagos parciales.
- Actualizar saldos sin sobreaplicar pagos ni facturas.
- Preparar la integración posterior con cartolas bancarias y factoring.

Al finalizar este bloque, el sistema podrá responder:

¿Qué facturas tiene un cliente?
¿Cuánto se ha pagado?
¿Cuánto permanece pendiente?
¿Está vencida?
¿Qué pagos cubren la factura?
¿A qué facturas fue aplicado un pago?
¿El pago fue recibido por la empresa o por factoring?

2. Resumen del bloque

Código	Actividad	Horas	Recurso	Cobertura actual	Resultado
F001	Clientes y contactos	4 h	Integraciones	Parcial	Maestro reutilizado y contactos históricos
F002	Registro o importación de facturas	6 h	Backend	Parcial	Alta manual e importación controlada

Código	Actividad	Horas	Recurso	Cobertura actual	Resultado
F003	Lista de facturas	4 h	Frontend	Parcial	Consulta filtrable y paginada
F004	Estado calculado de cobranza	8 h	Backend	Alta, con ajustes	Estado y saldo actualizados
F005	Registro de pagos de clientes	6 h	Backend	Parcial	Pago independiente de factura y banco
F006	Aplicación de pago a varias facturas	10 h	Backend	Alta, con ajustes	Un pago distribuido entre facturas
F007	Pagos parciales y múltiples	10 h	Backend	Alta, con ajustes	Una factura cubierta por varios pagos

El bloque suma **48 horas**.

Los criterios definidos en la Carta Gantt son:

- F001 reutiliza maestros existentes y conserva contacto histórico.
- F002 valida número, fechas, orden de compra, montos e impuestos.
- F003 filtra por cliente, estado, vencimiento y factoring.
- F004 calcula pendiente, parcial, pagada, vencida, cedida o anulada.
- F005 registra receptor, medio, fecha, monto y referencia.
- F006 distribuye un pago sin superar su monto disponible.
- F007 permite varios pagos sobre una factura y mantiene saldo pendiente.

3. Secuencia funcional

```

F001 Clientes y contactos
→ F002 Registro o importación de facturas
→ F003 Lista de facturas
→ F004 Estado calculado de cobranza
→ F005 Registro de pagos
→ F006 Un pago aplicado a varias facturas
→ F007 Una factura cubierta por varios pagos

```

En términos de dominio:

```

Cliente
├── Factura A
│   ├── Pago 1 parcial
│   └── Pago 2 parcial
├── Factura B
└── Pago 3

```

- └─ Aplicación a Factura A
- └─ Aplicación a Factura B

4. Separación entre factura, pago y movimiento bancario

Estos conceptos no deben fusionarse.

4.1 Factura

Representa la obligación de pago del cliente:

Cliente debe pagar \$1.000.000

4.2 PagoCliente

Representa un pago recibido, informado o registrado:

Cliente pagó \$700.000

El pago puede existir temporalmente sin estar asociado a una factura.

4.3 AplicacionPagoFactura

Indica cómo se distribuye el pago:

\$500.000 → Factura A
\$200.000 → Factura B

4.4 MovimientoBancario

Representa lo que efectivamente aparece en la cartola:

Ingreso bancario por \$700.000

La relación completa será:

MovimientoBancario
→ AplicacionMovimientoBancario
→ PagoCliente

- AplicacionPagoFactura
- FacturaVenta

La vinculación bancaria se desarrolla formalmente en F009 y B010. F005 no debe exigir obligatoriamente una cartola para registrar el pago.

5. Cobertura actual de modelos

5.1 Cliente

El modelo existente `extintores.Cliente` ya contiene:

- Nombre.
- RUT.
- Correo.
- Dirección.
- Comuna.
- Teléfono.
- Contacto.
- Información del último servicio.

No debe crearse un segundo modelo de cliente en Administración.

5.2 FacturaVenta

El modelo actual contiene:

- Cliente.
- Número.
- Orden de compra.
- Fecha de emisión.
- Fecha de vencimiento.
- Contacto.
- Correo.
- Teléfono.
- Neto.
- Tasa IVA.
- IVA.
- Total.
- Estado documental.
- Estado de cobranza.
- Observaciones.

También calcula:

- Monto pagado.
- Saldo pendiente.
- Días de atraso.

- Estado de cobranza.

5.3 PagoCliente

El modelo distingue:

- Cliente.
- Fecha.
- Monto.
- Receptor.
- Forma de pago.
- Referencia.
- Operación de factoring.
- Estado.
- Observaciones.

5.4 AplicacionPagoFactura

El modelo ya permite relacionar:

PagoCliente → FacturaVenta

y valida:

- Que la factura pertenezca al mismo cliente.
- Que las aplicaciones no superen el pago.
- Que los pagos válidos no superen el total de la factura.

6. Ajustes principales detectados

6.1 Contactos múltiples

El modelo `Cliente` solo admite un contacto principal mediante campos simples.

Para facturación y cobranza se necesitan varios contactos:

- Administración.
- Contabilidad.
- Pago de proveedores.
- Compras.
- Responsable técnico.
- Representante legal.

Se propone crear `ContactoCliente`.

6.2 Estado de factura

`NOTA_CREDITO` aparece actualmente como estado documental de una factura.

Sin embargo, una nota de crédito es un documento relacionado, no un estado equivalente a “emitida” o “anulada”.

La primera fase puede conservar el campo para no romper migraciones, pero la arquitectura correcta es crear una entidad separada:

```
NotaCreditoVenta
```

6.3 Validación de montos

El modelo actual restringe que el total sea positivo, pero no valida explícitamente:

```
IVA = neto × tasa IVA  
total = neto + IVA
```

Esta regla debe implementarse en formularios y servicios.

6.4 Estados persistidos

`estado_cobranza` se guarda en la base de datos, pero también se calcula.

Esto puede generar estados obsoletos si:

- Se aplica un pago.
- Se anula una aplicación.
- Se vence una factura durante la noche.
- Se crea o cierra una operación de factoring.

Debe existir un servicio único de actualización.

6.5 Validación mediante `clean()`

La lógica de `AplicacionPagoFactura.clean()` no se ejecuta automáticamente con cualquier llamada a `save()`.

Las aplicaciones deberán crearse mediante servicios que llamen:

```
aplicacion.full_clean()
```

antes de guardar.

F001 — Clientes y contactos

1. Objetivo

Reutilizar el maestro actual de clientes y ampliarlo para soportar contactos diferenciados para facturación, cobranza y pagos.

No se duplicará `Cliente`.

2. Modelos involucrados

Existente:

`extintores.Cliente`

Nuevo:

`administracion.ContactoCliente`

3. Modelo propuesto

```
class ContactoCliente(ModeloAuditoria):
    class Tipo(models.TextChoices):
        GENERAL = "GENERAL", "Contacto general"
        FACTURACION = "FACTURACION", "Facturación"
        COBRANZA = "COBRANZA", "Cobranza"
        PAGOS = "PAGOS", "Pago de proveedores"
        COMPRAS = "COMPRAS", "Compras"
        REPRESENTANTE = "REPRESENTANTE", "Representante legal"
        OTRO = "OTRO", "Otro"

    cliente = models.ForeignKey(
        "extintores.Cliente",
        on_delete=models.CASCADE,
        related_name="contactos_administrativos",
    )
    nombre = models.CharField(max_length=150)
    cargo = models.CharField(max_length=100, blank=True)
    tipo = models.CharField(
        max_length=20,
        choices=Tipo.choices,
        default=Tipo.GENERAL,
    )
```

```
correo = models.EmailField(blank=True)
telefono = models.CharField(max_length=40, blank=True)
principal = models.BooleanField(default=False)
activo = models.BooleanField(default=True)
observaciones = models.TextField(blank=True)
```

4. Ajustes recomendados a Cliente

Se recomienda incorporar:

```
activo = models.BooleanField(default=True)
```

También conviene normalizar y validar:

- RUT.
- Correo.
- Teléfono.
- Nombre.

No se recomienda establecer inmediatamente `rut` como único sin revisar primero los datos existentes, porque podría haber:

- RUT vacíos.
- Registros duplicados.
- Formatos diferentes.
- Sucursales registradas como clientes independientes.

5. Conservación del contacto histórico

Aunque el cliente cambie posteriormente de contacto, una factura ya emitida debe conservar:

- Contacto utilizado.
- Correo.
- Teléfono.

Por eso `FacturaVenta` debe seguir almacenando estos campos como snapshot:

```
FacturaVenta.contacto
FacturaVenta.correo
FacturaVenta.telefono
```

El flujo será:

Seleccionar contacto del cliente
→ copiar datos a la factura
→ conservarlos aunque el contacto cambie

6. Casos de uso

- Consultar clientes.
 - Ver contactos.
 - Agregar contacto.
 - Editar contacto.
 - Desactivar contacto.
 - Definir contacto principal.
 - Seleccionar contacto de facturación.
 - Seleccionar contacto de cobranza.
 - Consultar facturas del cliente.
 - Consultar pagos del cliente.
-

7. Reglas de negocio

1. El cliente se mantiene en la aplicación `extintores`.
 2. Un cliente puede tener múltiples contactos.
 3. Un contacto puede tener uno o varios usos.
 4. Solo debe existir un contacto principal activo por tipo.
 5. Los contactos inactivos no aparecen en nuevas facturas.
 6. Las facturas históricas conservan su snapshot.
 7. Desactivar un contacto no modifica facturas existentes.
 8. Los clientes con facturas no se eliminan físicamente.
 9. El RUT debe normalizarse antes de buscar duplicados.
 10. Un cliente inactivo permanece visible en documentos históricos.
-

8. URLs

```
/administracion/clientes/  
/administracion/clientes/<pk>/  
/administracion/clientes/<pk>/contactos/  
/administracion/clientes/<pk>/contactos/nuevo/  
/administracion/contactos/<pk>/editar/  
/administracion/contactos/<pk>/desactivar/
```

La edición integral del cliente puede continuar utilizando las vistas de la aplicación `extintores`.

9. Vistas

```
ClienteAdministracionListView
ClienteAdministracionDetailView
ContactoClienteCreateView
ContactoClienteUpdateView
ContactoClienteDesactivarView
ContactoClientePrincipalView
```

10. Template de detalle

Secciones:

Datos generales

- Nombre.
- RUT.
- Dirección.
- Comuna.
- Estado.

Contactos

Nombre	Tipo	Correo	Teléfono	Principal	Estado
--------	------	--------	----------	-----------	--------

Resumen financiero

- Facturas pendientes.
- Saldo vencido.
- Pagos sin aplicar.
- Operaciones de factoring.

Los indicadores serán completados durante el resto del módulo.

11. Permisos

```
view_cliente_administracion
change_cliente_administracion
view_contactocliente
add_contactocliente
change_contactocliente
desactivar_contactocliente
```

12. Auditoría

Registrar:

- Alta de contacto.
 - Cambio de correo.
 - Cambio de teléfono.
 - Cambio de tipo.
 - Definición como principal.
 - Desactivación.
-

13. Pruebas

- Crear varios contactos.
 - Definir principal.
 - Impedir dos principales del mismo tipo.
 - Desactivar principal.
 - Seleccionar contacto en factura.
 - Conservar snapshot.
 - Ocultar contactos inactivos.
 - Consultar historial del cliente.
 - Respetar permisos.
-

14. Criterios de aceptación

- No existe un segundo maestro de clientes.
 - Un cliente puede tener varios contactos.
 - Se distingue facturación de cobranza.
 - Las facturas conservan contacto histórico.
 - Los contactos inactivos no se utilizan en documentos nuevos.
 - La actividad queda auditada.
-

15. Evaluación de esfuerzo

Las 4 horas son suficientes para integrar el maestro existente y crear un CRUD sencillo de contactos si ya están implementados los componentes transversales.

La depuración de RUT duplicados debe estimarse aparte si los datos actuales contienen inconsistencias.

F002 — Registro o importación de facturas

1. Objetivo

Permitir registrar facturas manualmente o cargarlas desde un archivo estructurado, validando datos, fechas, impuestos y duplicados.

2. Modelos involucrados

Principal:

FacturaVenta

Relacionados:

Cliente
ContactoCliente
AdjuntoAdministracion

Para importación:

ImportacionFacturaVenta
ErrorImportacionFacturaVenta

3. Alta manual

Campos:

- Cliente.
- Número de factura.
- Orden de compra.
- Fecha de emisión.
- Fecha de vencimiento.
- Contacto.
- Correo.
- Teléfono.
- Neto.
- Tasa de IVA.
- IVA.
- Total.
- Estado documental.
- Observaciones.
- Archivo PDF o XML, opcional.

4. Cálculo monetario

Para una factura afecta:

```
IVA = redondear(neto × tasa_iva, 2)
total = neto + IVA
```

Para una factura exenta:

```
tasa_iva = 0
IVA = 0
total = neto
```

No debe confiarse exclusivamente en cálculos de JavaScript.

El backend debe recalcular y comparar.

5. Ajustes recomendados al modelo

Agregar:

```
class TipoTributario(models.TextChoices):
    AFECTA = "AFECTA", "Afecta a IVA"
    EXENTA = "EXENTA", "Exenta"
```

Campo:

```
tipo_tributario = models.CharField(
    max_length=10,
    choices=TipoTributario.choices,
    default=TipoTributario.AFECTA,
)
```

También conviene agregar:

```
moneda = models.CharField(max_length=3, default="CLP")
```

Para una fase futura con monedas extranjeras podrán incorporarse:

- Tipo de cambio.
- Total en moneda base.
- Fecha de conversión.

6. Validaciones

1. El número es obligatorio.
 2. El número no puede repetirse.
 3. La fecha de vencimiento no puede ser anterior a la emisión.
 4. El neto no puede ser negativo.
 5. El IVA no puede ser negativo.
 6. El total debe ser positivo.
 7. El total debe coincidir con neto más IVA.
 8. En una factura exenta, IVA debe ser cero.
 9. El cliente debe estar activo.
 10. La factura debe conservar el contacto utilizado.
 11. El correo debe tener formato válido.
 12. Una factura anulada no recibe nuevas aplicaciones.
 13. Una factura con pagos no puede anularse sin reversar sus aplicaciones.
 14. El número no podrá modificarse libremente después de tener pagos o factoring.
-

7. Nota de crédito

Se recomienda crear:

```
class NotaCreditoVenta(ModeloAuditoria):
    factura = models.ForeignKey(
        FacturaVenta,
        on_delete=models.PROTECT,
        related_name="notas_credito",
    )
    numero = models.PositiveIntegerField(unique=True)
    fecha_emision = models.DateField()
    motivo = models.TextField()
    neto = models.DecimalField(...)
    iva = models.DecimalField(...)
    total = models.DecimalField(...)
    anulada = models.BooleanField(default=False)
```

Esto es preferible a utilizar `NOTA_CREDITO` como estado de la factura.

En este bloque puede dejarse únicamente el esqueleto si la emisión de notas de crédito no forma parte del MVP inmediato.

8. Importación de facturas

Formatos iniciales:

CSV
XLSX

Columnas esperadas:

- RUT cliente.
- Número factura.
- Fecha emisión.
- Fecha vencimiento.
- Orden de compra.
- Neto.
- IVA.
- Total.
- Contacto.
- Correo.
- Observaciones.

9. Modelos de importación

```
class ImportacionFacturaVenta(ModeloAuditoria):
    class Estado(models.TextChoices):
        PENDIENTE = "PENDIENTE", "Pendiente"
        VALIDADA = "VALIDADA", "Validada"
        PROCESADA = "PROCESADA", "Procesada"
        CON_ERRORES = "CON_ERRORES", "Con errores"
        ANULADA = "ANULADA", "Anulada"

    archivo = models.FileField(...)
    nombre_archivo = models.CharField(max_length=255)
    sha256 = models.CharField(max_length=64, unique=True)
    estado = models.CharField(...)
    total_filas = models.PositiveIntegerField(default=0)
    total_validas = models.PositiveIntegerField(default=0)
    total_importadas = models.PositiveIntegerField(default=0)
    total_errores = models.PositiveIntegerField(default=0)
```

Y:

```
class ErrorImportacionFacturaVenta(ModeloAuditoria):
    importacion = models.ForeignKey(...)
    fila = models.PositiveIntegerField()
    campo = models.CharField(max_length=100, blank=True)
    codigo = models.CharField(max_length=50)
    mensaje = models.TextField()
    datos_fila = models.JSONField(default=dict)
```

10. Flujo de importación

```
Cargar archivo
→ calcular hash
→ leer filas
→ normalizar RUT
→ localizar cliente
→ validar factura
→ detectar duplicados
→ previsualizar
→ confirmar
→ crear facturas
```

La previsualización no debe crear facturas.

11. Cliente no encontrado

La política inicial recomendada es:

```
Cliente no encontrado
→ error bloqueante de la fila
```

No se crearán clientes automáticamente durante la importación, porque podrían generarse:

- Duplicados.
 - RUT incorrectos.
 - Registros incompletos.
-

12. URLs

```
/administracion/facturas/nueva/
/administracion/facturas/importar/
/administracion/facturas/importaciones/
/administracion/facturas/importaciones/<pk>/
/administracion/facturas/importaciones/<pk>/validar/
/administracion/facturas/importaciones/<pk>/confirmar/
```

13. Servicios

```
services/facturas/
└─ registro.py
```



```
|— importacion.py
|— validacion.py
|— calculos.py
└─ duplicados.py
```

Funciones:

```
calcular_totales_factura()
registrar_factura()
validar_archivo_facturas()
confirmar_importacion_facturas()
anular_factura()
```

14. Formularios

```
FacturaVentaForm
ImportacionFacturaVentaForm
ConfirmacionImportacionFacturaForm
```

La selección del contacto deberá completar automáticamente:

- Nombre.
- Correo.
- Teléfono.

Estos campos podrán ajustarse antes de guardar la factura.

15. Templates

```
templates/administracion/facturas/
|— form.html
|— importar.html
|— previsualizacion.html
|— errores_importacion.html
└─ confirmar_importacion.html
```

16. Permisos

```
add_facturaventa
change_facturaventa
importar_facturaventa
```

```
anular_facturaventa  
view_importacionfacturaventa
```

17. Auditoría

Registrar:

- Alta manual.
 - Importación.
 - Cambio de cliente.
 - Cambio de monto.
 - Cambio de fecha.
 - Anulación.
 - Documento adjunto.
 - Usuario y origen.
-

18. Pruebas

- Factura afecta válida.
 - Factura exenta válida.
 - IVA incorrecto.
 - Total incorrecto.
 - Fecha de vencimiento inválida.
 - Número duplicado.
 - Cliente inexistente.
 - Cliente inactivo.
 - Importación CSV.
 - Importación XLSX.
 - Doble importación.
 - Confirmación atómica.
 - Conservación de contacto histórico.
 - Anulación con pagos existentes.
-

19. Criterios de aceptación

- La factura puede registrarse manualmente.
 - Se validan número, fechas, montos e impuestos.
 - Puede importarse un archivo.
 - La importación se previsualiza antes de confirmar.
 - No se crean clientes automáticamente.
 - No se crean facturas duplicadas.
 - El contacto queda congelado como snapshot.
 - La operación queda auditada.
-

20. Evaluación de esfuerzo

Las 6 horas permiten un registro manual completo o una importación básica, pero implementar ambos flujos, errores por fila y pruebas puede requerir mayor esfuerzo.

Para respetar la Gantt se recomienda:

Primera iteración:
Registro manual completo

Segunda iteración:
Importación CSV/XLSX reutilizando el motor de B003

F003 — Lista de facturas

1. Objetivo

Permitir consultar las facturas de venta y detectar rápidamente:

- Pendientes.
- Parciales.
- Pagadas.
- Vencidas.
- Cedidas a factoring.
- Anuladas.

2. URL

```
/administracion/facturas/
```

Nombre:

```
administracion:factura_list
```

3. Vista

```
class FacturaVentaListView(  
    LoginRequiredMixin,  
    PermissionRequiredMixin,  
    ListView,
```

```
    ):  
        model = FacturaVenta  
        permission_required = "administracion.view_facturaventa"  
        paginate_by = 50
```

La lógica de consulta se ubicará en:

```
selectors/facturas.py
```

4. Filtros

Cliente

- Cliente.
- RUT.
- Cliente activo o inactivo.

Documento

- Número de factura.
- Orden de compra.
- Estado documental.

Fechas

- Emisión desde/hasta.
- Vencimiento desde/hasta.

Cobranza

- Pendiente.
- Parcial.
- Pagada.
- Vencida.
- Cedida a factoring.
- Anulada.
- Con saldo.
- Sin saldo.

Factoring

- No cedida.
- Cedida.
- Con recurso.
- Sin recurso.
- Operación activa.
- Operación cerrada.

Monto

- Total mínimo.
- Total máximo.
- Saldo mínimo.
- Saldo máximo.

Texto libre

- Cliente.
- RUT.
- Número.
- Orden de compra.
- Contacto.
- Correo.
- Observaciones.

5. QuerySet

La consulta deberá anotar:

```
monto_pagado_db  
saldo_pendiente_db  
cantidad_pagos
```

y precargar:

```
.select_related("cliente")
```

La operación de factoring podrá precargarse con cuidado mediante:

```
.select_related("operacion_factoring")
```

No debe llamarse `monto_pagado` para cada fila si esto genera consultas repetidas.

6. Columnas

Columna	Contenido
Número	Folio de factura
Cliente	Nombre y RUT
Emisión	Fecha
Vencimiento	Fecha

Columna	Contenido
Total	Total factura
Pagado	Aplicaciones válidas
Saldo	Pendiente
Estado	Cobranza calculada
Factoring	Empresa o estado
Acciones	Ver, cobrar, aplicar pago

7. Resumen del filtro

Mostrar:

- Cantidad de facturas.
- Total facturado.
- Total pagado.
- Saldo pendiente.
- Saldo vencido.
- Total cedido a factoring.

8. Ordenamiento

- Emisión reciente.
- Próxima a vencer.
- Más vencida.
- Mayor saldo.
- Cliente.
- Número.
- Estado.

Por defecto:

```
fecha_emision descendente
numero descendente
```

9. Acciones rápidas

- Ver factura.
- Registrar pago.
- Aplicar pago existente.
- Registrar gestión de cobranza.
- Ver factoring.

- Anular, con permiso.
-

10. Template

```
templates/administracion/facturas/  
├─ list.html  
├─ _filtros.html  
├─ _tabla.html  
└─ _resumen.html
```

11. Pruebas

- Filtrar por cliente.
 - Filtrar por estado.
 - Filtrar vencidas.
 - Filtrar cedidas.
 - Buscar por número.
 - Buscar por orden de compra.
 - Calcular saldo.
 - Calcular resumen.
 - Pagar conservar filtros.
 - Evitar consultas N+1.
 - Respetar permisos.
-

12. Criterios de aceptación

- Las facturas se listan y filtran.
 - El estado mostrado es consistente.
 - Los saldos son correctos.
 - Se identifican facturas vencidas.
 - Se distinguen facturas cedidas.
 - Los filtros se mantienen al paginar.
 - La consulta es eficiente.
-

F004 — Estado calculado de cobranza

1. Objetivo

Determinar automáticamente la situación financiera actual de cada factura.

Estados:

```
PENDIENTE
VENCIDA
PARCIAL
PAGADA
CEDIDA_FACTORING
ANULADA
```

El modelo ya contiene esta enumeración y una propiedad de cálculo.

2. Orden de evaluación recomendado

1. Documento anulado
2. Factura cedida con operación activa
3. Total pagado
4. Pago parcial
5. Vencimiento
6. Pendiente

Pseudológica:

```
if factura.estado_documental == ANULADA:
    return ANULADA

if tiene_factoring_activo:
    return CEDIDA_FACTORING

if monto_pagado >= total:
    return PAGADA

if monto_pagado > 0:
    return PARCIAL

if fecha_vencimiento and hoy > fecha_vencimiento:
    return VENCIDA

return PENDIENTE
```

3. Observación sobre factoring

`CEDIDA_FACTORING` indica que la cobranza fue transferida al factor.

No significa que la factura esté impaga o pagada desde el punto de vista económico.

Por ello, además del estado principal, la interfaz debería mostrar información complementaria:

Cedida a factoring
Cliente pagó al factor: Sí/No
Factor liquidó a empresa: Sí/No

Esto evita que un único estado intente representar todo el ciclo de factoring.

4. Monto pagado

Solo deben sumar como pago válido las aplicaciones con estado:

APLICADA

La lógica existente ya excluye:

- PAGO_ERRONEO_FACTORING.
- ANULADA.
- PENDIENTE.

La propiedad actual suma únicamente aplicaciones `APLICADA`.

5. Estado del pago

`PagoCliente.estado` también debe calcularse:

Condición	Estado
Pago anulado	ANULADO
Sin aplicaciones válidas	REGISTRADO
Aplicado parcialmente	PARCIAL
Aplicado completamente	APLICADO

Se agregará:

```
@property
def estado_calculado(self):
    ...
```

y:

```
def actualizar_estado(self, guardar=True):
    ...
```

6. Servicio central

```
services/cobranza/estados.py
```

Funciones:

```
actualizar_estado_factura()  
actualizar_estado_pago()  
actualizar_estados_afectados()  
actualizar_facturas_vencidas()
```

Después de crear o anular una aplicación:

```
Actualizar pago  
→ actualizar factura  
→ registrar auditoría
```

7. Actualización diaria

Una factura puede vencer sin que ningún usuario la modifique.

Se necesita un comando:

```
python manage.py actualizar_estados_cobranza
```

Este comando deberá:

- Buscar facturas emitidas.
- Excluir pagadas y anuladas.
- Recalcular vencidas.
- Actualizar solo estados modificados.
- Registrar un resumen.

Podrá ejecutarse diariamente mediante cron o el planificador utilizado por el proyecto.

8. Protección del campo

`estado_cobranza` no debe aparecer como campo editable en formularios normales.

Solo podrá modificarse mediante:

```
servicios de dominio  
comandos de actualización  
migraciones controladas
```

9. Concurrencia

Al aplicar pagos:

```
FacturaVenta.objects.select_for_update()  
PagoCliente.objects.select_for_update()
```

Esto evita que dos usuarios sobreapliquen la misma factura o pago.

10. Pruebas

- Factura pendiente.
 - Factura vencida.
 - Factura parcial.
 - Factura pagada.
 - Factura anulada.
 - Factura cedida.
 - Aplicación anulada.
 - Pago erróneo sobre factoring.
 - Factura que vence al cambiar de día.
 - Actualización diaria.
 - Aplicaciones concurrentes.
 - Estado de pago parcial y completo.
-

11. Criterios de aceptación

- El estado no se edita manualmente.
 - El estado coincide con pagos, vencimiento y factoring.
 - Las aplicaciones anuladas no suman.
 - Los pagos erróneos de factoring no pagan la factura.
 - Los estados vencidos se actualizan diariamente.
 - Los cambios quedan auditados.
 - No existen sobreaplicaciones concurrentes.
-

F005 — Registro de pagos de clientes

1. Objetivo

Registrar un pago de cliente independientemente de su posterior distribución entre facturas y de su conciliación bancaria.

La tarea debe capturar:

- Cliente.
- Receptor.
- Forma de pago.
- Fecha.
- Monto.
- Referencia.

2. Modelo involucrado

PagoCliente

El modelo ya contempla receptores:

EXTINTORES_REAL
FACTORING

y medios:

TRANSFERENCIA
DÉBITO
EFECTIVO
CHEQUE
OTRO

También dispone de estados de aplicación.

3. Formulario

Campos:

- Cliente.
- Fecha.
- Monto.
- Receptor.

- Forma de pago.
- Referencia.
- Operación de factoring, cuando corresponda.
- Observaciones.
- Comprobante opcional.

El estado no será editable.

4. Reglas del receptor

Receptor Extintores Real

```
operacion_factoring = vacío
```

salvo que el pago corresponda a una factura cedida pagada erróneamente a la empresa. En ese caso la relación podrá determinarse posteriormente durante la aplicación.

Receptor Factoring

Debe requerirse:

```
operacion_factoring
```

y el cliente del pago debe coincidir con el cliente de la factura cedida.

5. Validación propuesta

```
def clean(self):
    errors = {}

    if self.receptor == self.Receptor.FACTORING:
        if not self.operacion_factoring_id:
            errors["operacion_factoring"] = (
                "Debe indicar la operación de factoring."
            )

    if self.operacion_factoring_id:
        factura = self.operacion_factoring.factura
        if factura.cliente_id != self.cliente_id:
            errors["cliente"] = (
                "El cliente no coincide con la operación de factoring."
            )
```

```
if errors:
    raise ValidationError(errors)
```

6. Detección de duplicados

Se recomienda agregar:

```
fingerprint = models.CharField(
    max_length=64,
    blank=True,
)
```

Base posible:

```
cliente
+ fecha
+ monto
+ receptor
+ forma de pago
+ referencia
```

La referencia podrá ser obligatoria para transferencias, cheques y débitos cuando esté disponible.

Los pagos duplicados posibles deben generar advertencia, no necesariamente bloqueo automático.

7. Flujo

```
Registrar pago
→ estado REGISTRADO
→ opcionalmente aplicar a facturas
→ estado PARCIAL o APLICADO
→ posteriormente vincular con movimiento bancario
```

8. URLs

```
/administracion/pagos/
/administracion/pagos/nuevo/
/administracion/pagos/<pk>/
/administracion/pagos/<pk>/editar/
```

```
/administracion/pagos/<pk>/anular/  
/administracion/clientes/<cliente_id>/pagos/nuevo/
```

9. Vistas

```
PagoClienteListView  
PagoClienteDetailView  
PagoClienteCreateView  
PagoClienteUpdateView  
PagoClienteAnularView
```

La edición se restringirá cuando el pago tenga aplicaciones o conciliación bancaria.

10. Servicio

```
services/pagos/registro.py
```

Funciones:

```
registrar_pago()  
modificar_pago()  
anular_pago()  
buscar_posibles_duplicados()
```

11. Pantalla de detalle

Mostrar:

Pago

- Cliente.
- Fecha.
- Monto.
- Receptor.
- Medio.
- Referencia.
- Estado.

Distribución

- Monto aplicado.
- Saldo sin aplicar.

- Facturas vinculadas.

Banco

- Movimientos bancarios vinculados.
- Monto conciliado.
- Saldo bancario pendiente.

Factoring

- Operación relacionada.
 - Estado de la operación.
-

12. Anulación

No se eliminará el pago.

Para anular:

1. Deben anularse o reversarse aplicaciones activas.
 2. Deben reversarse aplicaciones bancarias.
 3. Debe indicarse motivo.
 4. Se cambia estado a **ANULADO**.
 5. Se registra auditoría.
-

13. Permisos

```
view_pagocliente  
add_pagocliente  
change_pagocliente  
anular_pagocliente
```

14. Pruebas

- Pago a empresa.
- Pago informado al factoring.
- Factoring sin operación.
- Operación de otro cliente.
- Monto cero.
- Monto negativo.
- Duplicado posible.
- Pago sin aplicaciones.
- Pago parcial.
- Pago completamente aplicado.
- Anulación sin aplicaciones.

- Impedir anulación con relaciones activas.
 - Auditoría.
-

15. Criterios de aceptación

- El pago puede registrarse sin factura inmediata.
 - El receptor queda identificado.
 - El pago a factoring exige operación válida.
 - El estado se calcula según aplicaciones.
 - El pago conserva saldo sin aplicar.
 - Los posibles duplicados son advertidos.
 - No se elimina físicamente.
 - La anulación es controlada y auditada.
-

F006 — Aplicación de pago a varias facturas

1. Objetivo

Distribuir un pago entre varias facturas del mismo cliente sin superar:

- El monto disponible del pago.
- El saldo pendiente de cada factura.

Ejemplo:

Pago recibido: \$1.000.000 CLP

Factura 1001: \$400.000

Factura 1002: \$350.000

Factura 1003: \$250.000

Resultado:

Pago completamente aplicado

Tres facturas pagadas

2. Modelo involucrado

AplicacionPagoFactura

El modelo ya valida que la factura pertenezca al cliente del pago y que no se excedan el pago ni la factura.

3. Servicio transaccional

`services/pagos/aplicaciones.py`

Función:

```
@transaction.atomic
def reemplazar_aplicaciones_pago(
    *,
    pago_id,
    aplicaciones,
    usuario,
    motivo="",
):
    pago = (
        PagoCliente.objects
        .select_for_update()
        .select_related("cliente")
        .get(pk=pago_id)
    )

    facturas_ids = [item["factura_id"] for item in aplicaciones]

    facturas = list(
        FacturaVenta.objects
        .select_for_update()
        .filter(pk__in=facturas_ids)
    )

    # Validar cliente.
    # Validar saldos.
    # Validar factoring.
    # Desactivar aplicaciones reemplazadas.
    # Crear nuevas aplicaciones.
    # Actualizar estados.
    # Registrar auditoría.
```

4. Reglas

1. El pago no puede estar anulado.
2. Todas las facturas deben pertenecer al cliente.
3. La factura debe estar emitida.
4. La factura no puede estar anulada.
5. El monto debe ser positivo.
6. La suma no puede superar el saldo del pago.

7. Cada aplicación no puede superar el saldo de la factura.
 8. Una factura puede recibir solo una fila por pago activo.
 9. Las aplicaciones anteriores no se eliminan físicamente.
 10. Las modificaciones requieren motivo cuando ya fueron conciliadas.
 11. El backend debe ejecutar `full_clean()`.
 12. Pago y facturas deben bloquearse en la misma transacción.
-

5. Restricción recomendada

Para evitar filas activas duplicadas entre el mismo pago y factura, puede agregarse una restricción condicional:

```
models.UniqueConstraint(  
    fields=["pago", "factura"],  
    condition=~Q(estado="ANULADA"),  
    name="uniq_adm_pago_factura_activa",  
)
```

La compatibilidad exacta dependerá del motor de base de datos.

6. Factura cedida a factoring

Pago recibido por la empresa

Cuando:

```
PagoCliente.receptor = EXTINTORES_REAL
```

y la factura tiene factoring activo, la aplicación debe crearse como:

```
PAGO_ERRONEO_FACTORING
```

No se suma al `monto_pagado` válido de la factura.

Además, se generará posteriormente una alerta:

```
PAGO_DIRECTO_FACTURA_CEDIDA
```

Pago recibido por el factor

Cuando:

```
PagoCliente.receptor = FACTORING
```

la operación de factoring del pago debe coincidir con la factura.

La aplicación podrá quedar como **APLICADA**, mientras el flujo de factoring registra la confirmación correspondiente.

7. Interfaz

URL:

```
/administracion/pagos/<pk>/aplicaciones/
```

Encabezado:

```
Pago #184
Cliente: Cliente X
Monto: $1.000.000 CLP
Aplicado: $600.000
Disponible: $400.000
```

Tabla:

Factura	Vencimiento	Total	Pagado	Saldo	Aplicar
1001	10-10-2026	\$500.000	\$300.000	\$200.000	\$200.000
1002	20-10-2026	\$300.000	\$100.000	\$200.000	\$200.000

8. Selección de facturas

Solo mostrar:

- Facturas del cliente.
- No anuladas.
- Con saldo pendiente.
- Ordenadas por vencimiento.
- Con indicador de factoring.

El sistema puede sugerir aplicar primero a la factura más antigua, pero no debe hacerlo automáticamente sin confirmación en esta tarea.

9. JavaScript

Funciones visuales:

- Seleccionar factura.
- Ingresar monto.
- Aplicar saldo completo.
- Distribuir visualmente.
- Mostrar disponible.
- Alertar excesos.
- Identificar factoring.

La validación definitiva permanece en Python.

10. Actualización de estados

Después de guardar:

Actualizar estado del pago

- actualizar cada factura afectada
- registrar auditoría

11. Auditoría

Registrar:

- Aplicaciones anteriores.
 - Aplicaciones nuevas.
 - Montos.
 - Facturas.
 - Estado anterior y nuevo del pago.
 - Estados anteriores y nuevos de las facturas.
 - Usuario.
 - Motivo.
-

12. Pruebas

- Un pago aplicado a dos facturas.
- Aplicación parcial.
- Aplicación completa.
- Superar pago.
- Superar factura.
- Factura de otro cliente.
- Factura anulada.
- Factura cedida pagada a empresa.

- Factura cedida pagada al factor.
 - Doble fila pago-factura.
 - Dos usuarios aplicando simultáneamente.
 - Rollback integral.
 - Estado actualizado.
-

13. Criterios de aceptación

- Un pago se distribuye entre varias facturas.
 - No se supera el pago.
 - No se supera ninguna factura.
 - Todas pertenecen al mismo cliente.
 - Se distingue factoring.
 - Los estados se recalculan.
 - Las aplicaciones anteriores permanecen en historial.
 - La operación es atómica.
 - Los cambios quedan auditados.
-

F007 — Pagos parciales y múltiples

1. Objetivo

Permitir que una factura sea pagada mediante varios pagos, manteniendo el saldo pendiente y el historial completo.

Ejemplo:

Factura: \$1.000.000 CLP

Pago 1: \$300.000

Pago 2: \$250.000

Pago 3: \$450.000

Evolución:

Después de Pago 1 → PARCIAL, saldo \$700.000

Después de Pago 2 → PARCIAL, saldo \$450.000

Después de Pago 3 → PAGADA, saldo \$0

2. Diferencia con F006

F006

Un pago → varias facturas

F007

Una factura → varios pagos

Ambas funcionalidades utilizan `AplicacionPagoFactura`, pero necesitan vistas y pruebas diferentes.

3. Vista desde la factura

URL:

`/administracion/facturas/<pk>/pagos/`

Mostrar:

- Total.
- Monto pagado.
- Saldo.
- Estado.
- Aplicaciones.
- Pagos disponibles del cliente.
- Factoring.

4. Tabla de pagos

Pago	Fecha	Receptor	Monto pago	Aplicado a factura	Estado
#101	01-10-2026	Empresa	\$300.000	\$300.000	Aplicada
#115	08-10-2026	Empresa	\$500.000	\$250.000	Aplicada
#130	15-10-2026	Empresa	\$450.000	\$450.000	Aplicada

5. Saldos

Para la factura:

$\text{saldo pendiente} = \text{total} - \text{aplicaciones válidas}$

Para cada pago:

$\text{saldo sin aplicar} = \text{monto pago} - \text{aplicaciones no anuladas}$

La aplicación no puede superar ninguno de los dos saldos.

6. Pago parcial

Una aplicación puede ser menor que:

- El saldo del pago.
- El saldo de la factura.

Ejemplo:

Pago disponible: \$500.000
Saldo factura: \$700.000
Aplicación: \$300.000

Resultado:

Pago mantiene \$200.000 sin aplicar
Factura mantiene \$400.000 pendiente

7. Aplicación desde la factura

El usuario podrá:

1. Abrir una factura.
2. Seleccionar un pago existente del mismo cliente.
3. Indicar el monto.
4. Confirmar.
5. Recalcular ambos saldos.

También podrá registrar un pago nuevo desde la factura, pero se crearán dos operaciones explícitas:

Registrar PagoCliente
→ crear AplicacionPagoFactura

No deben fusionarse de forma que se pierda la trazabilidad.

8. Pagos seleccionables

Solo mostrar:

- Pagos del mismo cliente.
 - No anulados.
 - Con saldo sin aplicar.
 - Compatibles con factoring.
 - Ordenados por fecha.
-

9. Reglas

1. Una factura admite varios pagos.
 2. Un pago admite varias facturas.
 3. Una aplicación debe ser positiva.
 4. No puede exceder saldo de factura.
 5. No puede exceder saldo de pago.
 6. Las aplicaciones anuladas no suman.
 7. Las aplicaciones erróneas de factoring no pagan la factura.
 8. Una factura pagada no admite nuevas aplicaciones.
 9. Una factura anulada no admite aplicaciones.
 10. Una aplicación conciliada no se altera sin reversa.
 11. El orden de pagos no cambia el resultado del saldo.
 12. Los cálculos deben utilizar `Decimal`.
-

10. Estado parcial

La factura queda `PARCIAL` cuando:

0 < monto pagado válido < total

El pago queda `PARCIAL` cuando:

0 < monto aplicado < monto del pago

Estos dos estados son independientes.

11. Anulación de aplicación

Al anular una aplicación:

```
Aplicación → ANULADA
Pago recupera saldo
Factura recupera saldo
Estados se recalculan
Auditoría se registra
```

No se elimina la fila.

Se recomienda agregar:

```
anulada_en = models.DateTimeField(null=True, blank=True)
anulada_por = models.ForeignKey(...)
motivo_anulacion = models.TextField(blank=True)
```

12. Concurrencia

El servicio debe bloquear:

```
PagoCliente
FacturaVenta
Aplicaciones activas relevantes
```

para evitar que dos usuarios utilicen simultáneamente el mismo saldo.

13. Servicio reutilizado

F007 reutilizará el mismo servicio de F006, pero expondrá una operación orientada a factura:

```
@transaction.atomic
def aplicar_pago_a_factura(
    *,
    factura_id,
    pago_id,
    monto,
    usuario,
):
    ...
```

Internamente puede llamar a una función común de validación.

14. URLs

```
/administracion/facturas/<pk>/pagos/  
/administracion/facturas/<pk>/aplicar-pago/  
/administracion/aplicaciones-pago/<pk>/anular/
```

15. Vistas

```
FacturaPagosView  
FacturaAplicarPagoView  
AplicacionPagoAnularView
```

16. Templates

```
templates/administracion/facturas/  
├─ pagos.html  
├─ aplicar_pago.html  
└─ anular_aplicacion.html
```

17. Permisos

```
view_aplicacionpagofactura  
add_aplicacionpagofactura  
anular_aplicacionpagofactura
```

18. Auditoría

Registrar:

- Factura.
- Pago.
- Monto.
- Saldo anterior de factura.
- Saldo nuevo.
- Saldo anterior del pago.
- Saldo nuevo.
- Estados anteriores y nuevos.
- Usuario.

- Motivo de anulación.
-

19. Pruebas

- Dos pagos parciales.
 - Tres pagos que completan factura.
 - Pago mayor que saldo de factura.
 - Factura mayor que saldo del pago.
 - Aplicación anulada.
 - Recuperación de saldos.
 - Factura pagada.
 - Factura parcial.
 - Pago parcial.
 - Pago completamente aplicado.
 - Factura cedida.
 - Pago de otro cliente.
 - Aplicaciones concurrentes.
 - Precisión decimal.
-

20. Criterios de aceptación

- Una factura admite múltiples pagos.
 - Los saldos se actualizan correctamente.
 - El estado parcial se determina automáticamente.
 - Una aplicación anulada revierte sus efectos.
 - No se superan los saldos.
 - Se respetan cliente y factoring.
 - Existe trazabilidad completa.
 - Las operaciones concurrentes son seguras.
-

7. Estructura de archivos del bloque

```
administracion/
├── models/
│   ├── clientes.py
│   └── facturacion.py
├── forms/
│   ├── contactos_cliente.py
│   ├── facturas.py
│   ├── importacion_facturas.py
│   ├── pagos.py
│   └── aplicaciones_pago.py
└── views/
```

```
|
| ├── clientes.py
| ├── facturas.py
| ├── importacion_facturas.py
| ├── pagos.py
| └── aplicaciones_pago.py
|
| ├── services/
| | ├── facturas/
| | | ├── registro.py
| | | ├── calculos.py
| | | ├── estados.py
| | | ├── importacion.py
| | | └── validacion.py
| | └── pagos/
| |   ├── registro.py
| |   ├── aplicaciones.py
| |   └── estados.py
|
| ├── selectors/
| | ├── clientes.py
| | ├── facturas.py
| | └── pagos.py
|
| ├── management/commands/
| | └── actualizar_estados_cobranza.py
|
| ├── templates/administracion/
| | ├── clientes/
| | ├── facturas/
| | └── pagos/
|
| ├── static/administracion/js/
| | ├── facturas.js
| | └── aplicaciones_pago.js
|
| └── tests/
|   ├── test_contactos_cliente.py
|   ├── test_registro_factura.py
|   ├── test_importacion_facturas.py
|   ├── test_lista_facturas.py
|   ├── test_estados_cobranza.py
|   ├── test_registro_pagos.py
|   ├── test_aplicacion_pago_multiple.py
|   └── test_pagos_parciales.py
```

8. Migraciones previstas

Migración 1 — Contactos

- Crear `ContactoCliente`.
- Evaluar campo `Cliente.activo`.
- Agregar restricciones para contacto principal.

Migración 2 — Factura

- Agregar tipo tributario.
- Agregar moneda.
- Revisar estado documental.
- Agregar validaciones e índices.

Migración 3 — Importación

- Crear `ImportacionFacturaVenta`.
- Crear `ErrorImportacionFacturaVenta`.

Migración 4 — Pago

- Agregar fingerprint, si se aprueba.
- Agregar índices por cliente, fecha y estado.

Migración 5 — Aplicaciones

- Agregar trazabilidad de anulación.
- Agregar restricción pago-factura activa.
- Agregar índices por pago, factura y estado.

Migración 6 — Nota de crédito

- Crear `NotaCreditoVenta`, cuando se incorpore al alcance.

9. Índices recomendados

FacturaVenta

```
models.Index(fields=["cliente", "estado_cobranza"])
models.Index(fields=["cliente", "fecha_vencimiento"])
models.Index(fields=["estado_documental", "estado_cobranza"])
```

PagoCliente

```
models.Index(fields=["cliente", "fecha"])
models.Index(fields=["cliente", "estado"])
models.Index(fields=["receptor", "fecha"])
```

AplicacionPagoFactura

```
models.Index(fields=["pago", "estado"])
models.Index(fields=["factura", "estado"])
```

ContactoCliente

```
models.Index(fields=["cliente", "tipo", "activo"])
```

10. Definición de terminado del bloque

F001–F007 se considerará terminado cuando:

- Se reutiliza `extintores.Cliente`.
 - Un cliente puede tener varios contactos.
 - Las facturas conservan contacto histórico.
 - Se pueden registrar facturas manualmente.
 - Se validan fechas, neto, IVA y total.
 - Existe un flujo de importación definido.
 - Las facturas pueden listarse y filtrarse.
 - El estado de cobranza se calcula automáticamente.
 - Las facturas vencidas se actualizan diariamente.
 - Se pueden registrar pagos sin factura inmediata.
 - Se distingue pago a empresa de pago a factoring.
 - Un pago puede distribuirse entre varias facturas.
 - Una factura puede recibir varios pagos.
 - No se supera el monto del pago.
 - No se supera el saldo de la factura.
 - Los pagos erróneos sobre factoring se identifican.
 - Los estados de pago y factura se recalculan.
 - Las aplicaciones anuladas conservan historial.
 - Las operaciones son atómicas.
 - Se controlan concurrencias.
 - Las pruebas automáticas pasan.
-

11. Riesgos del bloque

Riesgo	Impacto	Tratamiento
Duplicar maestro de clientes	Alto	Reutilizar <code>extintores.Cliente</code>
Perder contacto histórico	Alto	Snapshot en factura
IVA o total incorrecto	Alto	Recálculo en backend
Estado de cobranza obsoleto	Alto	Servicio y comando diario
Sobreaplicar un pago	Alto	Bloqueo y validación
Sobrepagar una factura	Alto	Bloqueo y validación
Aplicar pago de otro cliente	Alto	Restricción por cliente
Pago de factura cedida a empresa	Alto	Estado especial y alerta
Confundir pago con cartola	Alto	Entidades separadas
Editar aplicaciones sin historial	Alto	Anular y recrear
Dos usuarios aplican el mismo saldo	Alto	<code>select_for_update()</code>
Nota de crédito modelada como estado	Medio/alto	Entidad separada
Clientes duplicados por RUT	Alto	Normalización y depuración previa

12. Resultado consolidado

Al finalizar este bloque existirá el flujo:

```
Cliente existente
→ contacto de facturación
→ factura registrada
→ estado de cobranza calculado
→ pago registrado
→ pago distribuido
→ factura parcialmente o totalmente pagada
→ saldos actualizados
→ trazabilidad registrada
```

La salida habilita el siguiente bloque:

```
F008 – Excedentes y pagos sin aplicar
F009 – Vinculación con cartola bancaria
F010 – Gestión de cobranza
F011 – Alertas de vencimiento
```


F012 – Antigüedad de saldos
F013 – Reportes y exportación
F014 – Pruebas del módulo